

Structures de données et algorithmes

Chapitre 4 Les arbres

4 Les arbres

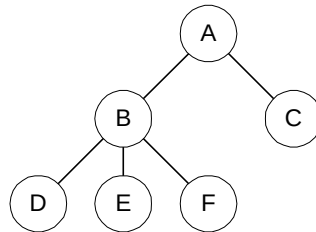
- Nous introduisons dans ce chapitre le TDA : arbres binaires
- Ce TDA est intéressant pour plusieurs raisons :
 - Les arbres de recherche binaire permettent d'insérer, de retrouver ou de supprimer les éléments $O(\log N)$ au lieu de $O(N)$.
 - L'algorithme de tri « Tree sort » est le plus rapide des algorithmes de tri conventionnel et il est basé sur un arbre de recherche binaire.
 - Pour représenter des files de priorité ou des files d'attente, l'arbre en épi est très bien adapté. Ce type d'arbre est aussi à la base d'un autre algorithme de tri efficace, le « Heap sort ».
 - Un autre aspect important est l'efficacité des arbres d'expression pour les constructions mathématiques. Ils permettent d'évaluer sans ambiguïté les expressions algébriques et de faire du calcul symbolique.

4 Les arbres

4.1 Terminologie des arbres

- Tous les arbres sont hiérarchiques de nature. Intuitivement, la hiérarchie signifie que dans un arbre existe une relation « père-fils » entre les nœuds. S'il existe un arc orienté de n vers m , alors n est le père de m et m est le fils de n .

Par exemple : B et C sont des fils de A et B est le père de D, E et F



3

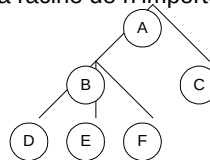
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- Les enfants du même parent sont dits frères.
- Dans un arbre, chaque nœud possède au plus un parent et il y a exactement un nœud, nommé racine de l'arbre, qui n'a pas de père. Ici A est la racine.
- Les nœuds qui ne possèdent pas de fils sont appelés feuille de l'arbre. Les feuilles de l'arbre montré ici sont C, D, E et F.
- La relation « père-fils » entre les nœuds d'un arbre peut être généralisée à la relation ancêtre et descendant. Dans cet exemple, A est un ancêtre de D et D est un descendant de A. Attention, ce ne sont pas tous les nœuds qui sont liés par cette relation, par exemple, les nœuds B et C ne sont pas reliés par cette relation. Par contre, la racine de n'importe quel arbre est l'ancêtre de tous les autres nœuds.



4

GPA665 - Automne 2002

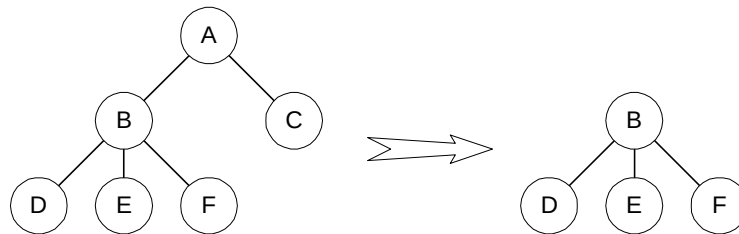
© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- Un sous-arbre d'un arbre est l'ensemble de n'importe quel nœud de l'arbre et tous ses descendants. Le sous-arbre du nœud n est le sous-arbre dont la racine est un fils de n .

Par exemple : le sous-arbre possédant B comme racine est un sous-arbre du nœud A



5

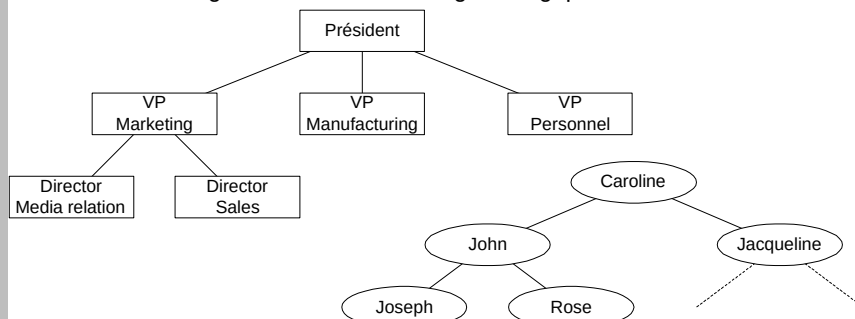
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- Puisque les arbres sont hiérarchique de nature, on peut alors les utiliser pour représenter l'information qui, elle-même est hiérarchique. Par exemple, une carte d'organisation ou un arbre généalogique.



6

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- Un arbre binaire est un arbre dans lequel chaque nœud possède au plus deux fils. L'arbre généalogique présenté plus tôt en est un bon exemple contrairement à la carte d'organisation (qu'on nomme arbre n -aire).
- Voici la définition formelle d'un arbre binaire :
 - Un ensemble T d'éléments, appelés nœuds, est un arbre binaire si, soit:
 - T est vide
 - T est partitionné en trois ensembles disjoints :
 - Un élément singulier r (sa racine)
 - Deux ensembles sont des arbres binaires, nommés sous-arbres gauche et droit de r
 - Ceci peut être schématisé par : T est un arbre binaire si, soit :
 - T ne possède aucun nœud
 - T est de la forme :
$$\begin{array}{c} r \\ \swarrow \quad \searrow \\ T_{\text{gauche}} \quad T_{\text{droit}} \end{array}$$

7

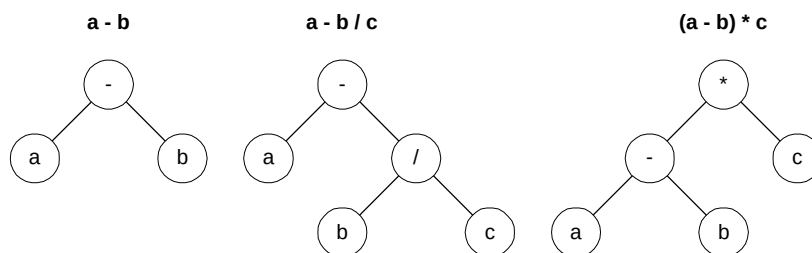
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- Considérer les exemples ci-dessous d'évaluations des expressions arithmétiques, pour voir comment sont utilisés les arbres binaires pour représenter les données dans une forme hiérarchique.



8

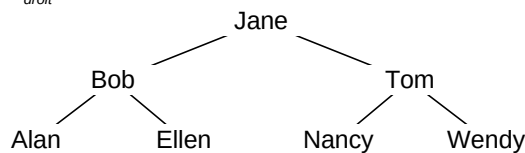
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- Un arbre de recherche binaire est un arbre binaire ordonné suivant les valeurs associées à ses nœuds. Pour chaque nœud n , un arbre de recherche binaire satisfait les trois propriétés suivantes :
 - La valeur du nœud n est plus grande que toutes les valeurs de son sous-arbre gauche (T_{gauche}).
 - La valeur du nœud n est plus petite que toutes les valeurs de son sous-arbre droit (T_{droit}).
 - T_{gauche} et T_{droit} sont tous les deux des arbres de recherche binaires.



- Comme on le remarque, ce type d'arbre organise les données de telle façon que la recherche d'un élément particulier devienne facile.

9

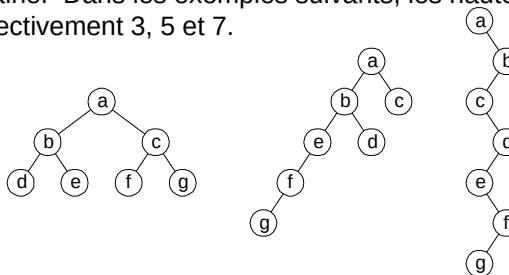
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- Les arbres apparaissent sous plusieurs formes. Dans l'exemple suivant, malgré qu'ils aient le même nombre de sommet (7), leurs structures sont complètement différentes.
- La hauteur d'un arbre est la distance entre la racine et la feuille la plus lointaine. Dans les exemples suivants, les hauteurs des arbres sont respectivement 3, 5 et 7.



10

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- La hauteur d'un arbre général T , en terme de niveau de nœuds, est maintenant définie comme suit :
 - Si T est vide alors sa hauteur est 0
 - Si T n'est pas vide, alors sa hauteur est égale au niveau maximum de ses nœuds
- Il est convenable d'utiliser une définition récursive pour calculer la hauteur d'un arbre binaire :
 - Si T est vide, alors sa hauteur est 0
 - Si T n'est pas vide alors la hauteur de T peut être définie comme un plus la hauteur du plus grand sous-arbre

$$\text{Hauteur}(T) = 1 + \max(\text{Hauteur}(T_{\text{gauche}}), \text{Hauteur}(T_{\text{droit}}))$$

11

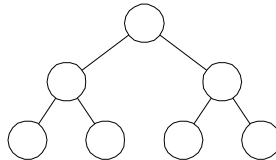
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

- Un arbre binaire plein de hauteur h possède toutes ses feuilles au niveau h , et tous les nœuds qui sont au niveau inférieur à h possèdent deux enfants. L'arbre suivant est un arbre plein de hauteur 3.



- Voici une définition convenable d'un arbre binaire plein :
 - Si T est vide, alors T est un arbre binaire plein de hauteur 0
 - Si T n'est pas vide et possède une hauteur $h > 0$, alors T est un arbre binaire plein si les deux sous-arbres de la racine sont des arbres binaires pleins de hauteur $h - 1$

12

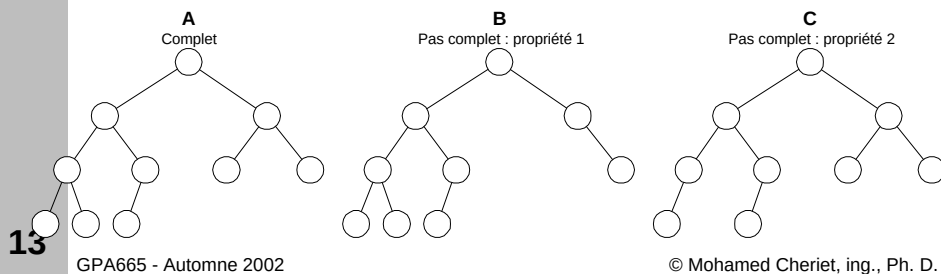
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.1 Terminologie des arbres

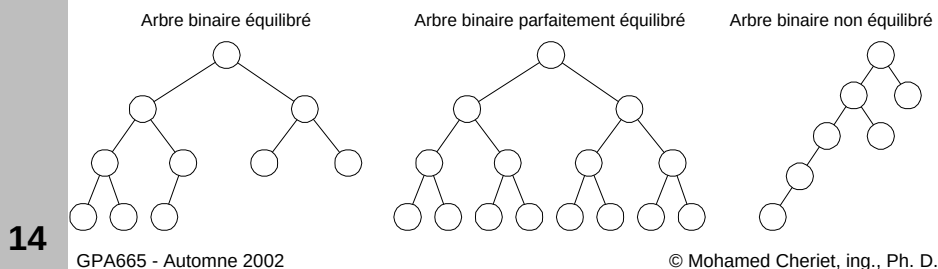
- Un arbre binaire complet de hauteur h est un arbre binaire qui est plein jusqu'au niveau $h - 1$, avec le niveau h rempli de gauche à droite.
- Plus formellement, un arbre binaire T de hauteur h est complet :
 1. Si tous les nœuds du niveau $h - 2$ et moins possèdent deux enfants
 2. Si un nœud possède un descendant droit et au niveau h , toutes les feuilles de son sous-arbre gauche sont de niveau h .
- *Noter que si un arbre binaire est plein, il est forcément complet*



4 Les arbres

4.1 Terminologie des arbres

- Un arbre binaire est équilibré si la hauteur du sous-arbre droit de n'importe quel nœud diffère de la hauteur du sous-arbre gauche d'au plus 1
- Un arbre binaire est parfaitement équilibré si le sous-arbre droit et le sous-arbre gauche de chaque nœud possèdent la même hauteur
- Noter qu'un arbre binaire complet est équilibré et qu'un arbre binaire plein est parfaitement équilibré. Inversement, les arbres équilibrés ne sont pas forcément complets.



4 Les arbres

4.2 Responsabilités du TDA arbres binaires

- Responsabilités globales
 - Rendre un arbre vide
 - Vérifier si un arbre est vide
 - Calculer la taille courante d'un arbre binaire
 - Calculer la hauteur courante d'un arbre binaire
 - Copier un arbre binaire dans un autre arbre binaire
 - Copier le sous-arbre gauche dans un autre arbre binaire
 - Copier le sous-arbre droit dans un autre arbre binaire
 - Appliquer les différents parcours d'un arbre

15

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.2 Responsabilités du TDA arbres binaires

- Responsabilités locales
 - Retrouver, insérer et supprimer un nœud d'un arbre binaire
- Un problème intéressant, comment peut-on placer des éléments dans un arbre binaire?
 - Une possibilité est de stocker les éléments de l'arbre selon l'ordre de leurs valeurs
 - Une deuxième possibilité pour stocker un élément est de spécifier sa position dans l'arbre

16

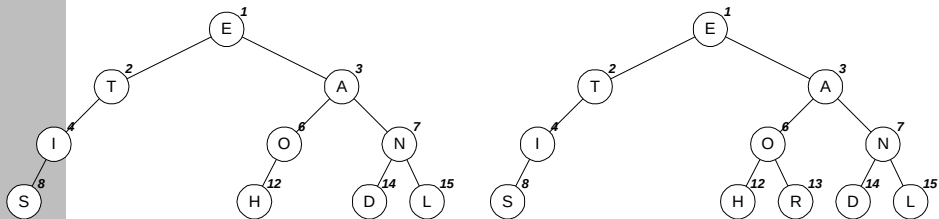
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.2 Responsabilités du TDA arbres binaires

- À partir de l'arbre de gauche suivant on désire insérer un nœud de valeur R à la position 13, qui donne l'arbre de droite



- Pour que l'insertion ait un sens, il faut que le parent de la position en question existe. Par exemple on ne peut insérer un élément dans la position 10 parce qu'il n'y a pas de nœud à la position 5

17

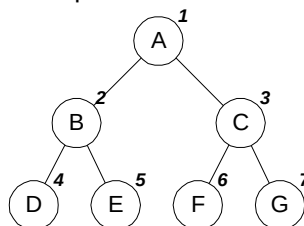
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.2 Responsabilités du TDA arbres binaires

- Une autre complication apparaît quand à la suppression d'un élément. Si un élément non feuille de l'arbre est supprimé, comment doit-on restructurer l'arbre? Imaginons que nous désirons enlever l'élément à la position 1 de l'arbre suivant.



- On ne sait pas si c'est B ou C qui sera la nouvelle racine, à moins qu'on connaisse la relation entre les éléments

18

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.3 Caractéristiques du TDA arbre binaire

- Dans un cadre général, les opérations d'insertion, de suppression et de consultation constituent le sujet de l'organisation des données.
- Les TDA tels que les piles et les files sont tous orientés suivant le critère position. Ainsi, les différentes opérations sont de la forme :
 - *Action* à la *i*-ème position dans la structure de données
Où l'action peut être l'insertion, la suppression et la consultation
- Dans ce chapitre, nous avons introduit un TDA orienté vers les valeurs et dont les opérations sont de la forme :
 - *Action* d'une donnée de valeur *x* dans la structure de données
où, encore une fois, l'action peut être une insertion, suppression ou consultation

4 Les arbres

4.4 Implantation d'un arbre binaire

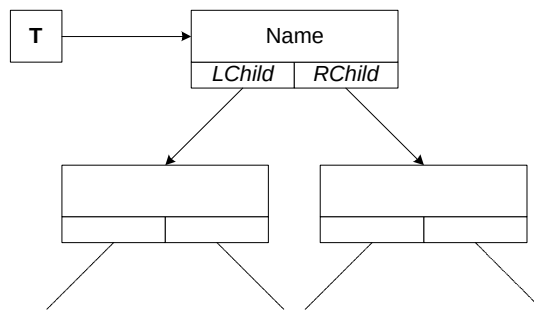
- Il y a deux façon possibles d'implanter les arbre binaires. Une possibilité qui utilise les pointeurs ou l'autre qui utilise les tableaux.
- Implantation basée sur les pointeurs

```
#define maxlength 20      /* longueur max d'un nom */
typedef struct _node {
    char Name[maxlength];
    _node *LChild;        /* Pointeur vers fils gauche */
    _node *RChild;        /* Pointeur vers fils droit */
} Node;

Node *T; /* Pointeur vers la racine de l'arbre */
```

4 Les arbres

4.4 Implantation d'un arbre binaire



21

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.4 Implantation d'un arbre binaire

- Implantation basée sur les tableaux. On utilise les indices du tableau pour indiquer les nœuds fils de l'arbre comme le spécifie la définition suivante :

```
#define maxnode 100      /* nombre max de noeuds */
#define maxlength 20     /* longueur max d'un nom */
struct {
    char Name[maxlength];
    int LChild;           /* Indice du fils gauche */
    int RChild;           /* Indice du fils droit */
} BinTree[maxnode];

int T, Free; /*Indice de la racine et de la liste libre*/
```

22

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.4 Implantation d'un arbre binaire

- L'indice T indique la racine de l'arbre.
- Comme l'arbre change à cause des insertions et suppressions, les nœuds peuvent ne pas être dans un ordre consécutif dans l'arbre. C'est pourquoi il faut établir une liste de nœuds disponibles. À chaque insertion et suppression il est important de consulter et mettre à jour cette liste.
- L'indice Free indique le début de la liste vide.

T	Free
1	8

Arbre		
	Name	LChild RChild
1	Jane	2 3
2	Bob	4 5
3	Tom	6 7
4	Alan	0 0
5	Ellen	0 0
6	Nancy	0 0
7	Wendy	0 0
8		0 9
9		0 10
10	...	...

23

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.5 Parcours des arbres binaires

- La définition d'un arbre binaire met en évidence sa nature récursive.
- Ainsi, en tenant compte de cette définition, on peut construire un algorithme de parcours récursif comme suit :

```

  Traverse(T)
  /* Parcours de l'arbre binaire T */
  {
    if (T n'est pas vide)
    {
      Traverse(sous-arbre gauche de T);
      Traverse(sous-arbre droit de T);
    }
  }

```

24

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.5 Parcours des arbres binaires

- Pour afficher le contenu de l'arbre pendant son parcours, il existe trois possibilités d'ordre d'affichage :
 - Avant de parcourir les deux sous-arbres de T
 - Après avoir parcouru le sous-arbre gauche de T , mais avant de parcourir le sous-arbre droit de T
 - Après avoir parcouru les deux sous-arbres de T
- Ces algorithmes de parcours sont nommés par conséquent
 - preorder
 - inorder
 - postorder

25

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.5 Parcours des arbres binaires

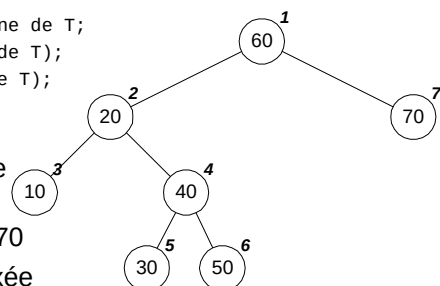
- L'algorithme de parcours preorder :

```
Preorder(T)
/* Parcours de l'arbre binaire T */
{
  if (T n'est pas vide) {
    Imprime la donnée de la racine de T;
    Preorder (sous-arbre gauche de T);
    Preorder (sous-arbre droit de T);
  }
}
```

- Le parcours preorder de l'arbre suivant donne

60 – 20 – 10 – 40 – 30 – 50 – 70

- Correspond à la notation préfixée



26

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.5 Parcours des arbres binaires

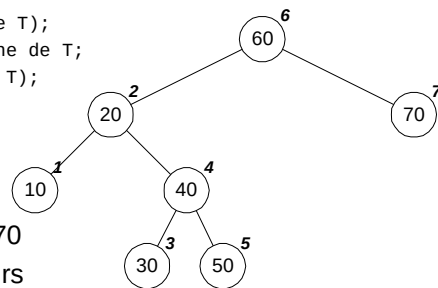
- L'algorithme de parcours inorder :

```
Inorder(T)
/* Parcours de l'arbre binaire T */
{
  if (T n'est pas vide) {
    Inorder (sous-arbre gauche de T);
    Imprime la donnée de la racine de T;
    Inorder (sous-arbre droit de T);
  }
}
```

- Le parcours inorder de l'arbre suivant donne

10 – 20 – 30 – 40 – 50 – 60 – 70

- Correspond à l'ordre des valeurs



27

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.5 Parcours des arbres binaires

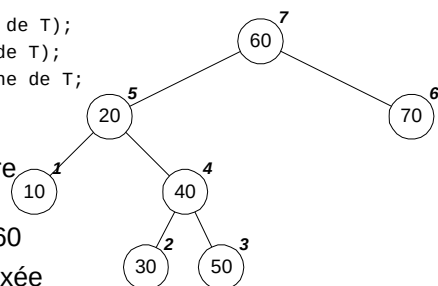
- L'algorithme de parcours postorder :

```
Postorder(T)
/* Parcours de l'arbre binaire T */
{
  if (T n'est pas vide) {
    Postorder (sous-arbre gauche de T);
    Postorder (sous-arbre droit de T);
    Imprime la donnée de la racine de T;
  }
}
```

- Le parcours postorder de l'arbre suivant donne

10 – 30 – 50 – 40 – 20 – 70 – 60

- Correspond à la notation postfixée



28

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.5 Parcours des arbres binaires

- On peut facilement implanter les trois algorithmes de parcours en langage C. L'implantation basée sur les pointeurs de l'algorithme de la fonction `inorder` apparaîtra comme suit :

```
void Inorder(Node *T)
/* Parcours de l'arbre binaire T */
{
    if (T) {
        Inorder(T->LChild);
        printf("%s\n", T->Name);
        Inorder(T->RChild);
    }
}
```

- On peut aussi envisager une fonction non récursive en se basant sur les piles (voir p.132, Notes de cours, M. Cheriet)

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Nous étudions dans cette section trois descendants du TDA arbres binaires :
 - Les arbres de recherche binaire. Ils sont ordonnés de telle façon que le temps moyen d'une insertion, suppression ou de recherche soit seulement $O(\log N)$. Ils sont notamment utilisés dans les situations où les insertions et les suppressions sont aléatoires.
 - Les arbres organisés en épi sont ordonnés d'une manière assez spéciale, de telle façon qu'ils soient commode pour :
 - Les pires cas de tri
 - Les files de priorité
 - Les files d'attente
 - Les arbres d'expression offrent une bonne façon de représenter une expression arithmétique dont les opérateurs sont binaires, c'est-à-dire qu'ils possèdent deux opérandes. Cependant, on peut facilement appliquer des opérations arithmétiques sur les expressions, telles que les dérivées symboliques ou les intégrales.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Les arbres de recherche binaire.
- On commence par une définition récursive de ces derniers :
 - Un arbre de recherche binaire T est une arbre binaire tel que T est vide ou :
 - Tout élément dans le sous-arbre gauche de T est inférieur à l'élément racine de T
 - Tout élément dans le sous-arbre droit de T est supérieur à l'élément racine de T
 - Les sous-arbres gauche et droit sont des arbres de recherche binaire
- *Remarque : les éléments dupliqués ne sont pas permis dans les arbre de recherche binaire*

31

GPA665 - Automne 2002

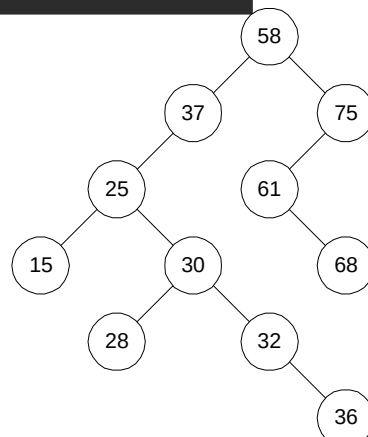
© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Voici un exemple d'arbre de recherche binaire
- Un parcours inorder d'un arbre de recherche binaire accède aux éléments dans un ordre croissant. Nous avons, pour l'affichage de cet exemple :

15 – 25 – 28 – 30 – 32 – 36 –
37 – 58 – 61 – 68 – 75



32

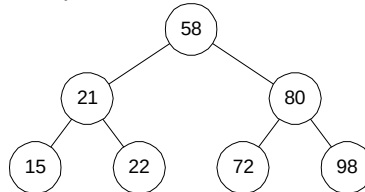
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Un arbre de recherche binaire est appelé ainsi parce que la recherche d'un élément se fait comme dans une recherche dichotomique lorsque l'arbre est dans une situation parfaitement équilibrée,.



- Les arbres de recherche binaires servent le même but que les listes (tableaux) ordonnées, ils donnent une structure permettant de trier les éléments dans l'ordre.
- Comme on va le voir, un arbre de recherche binaire est généralement plus efficace qu'une liste ordonnée pour l'insertion et la suppression d'éléments

33

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Les responsabilités locales d'un arbre de recherche binaire sont :
 - Insérer un élément
 - Supprimer un élément
 - Retrouver un élément
- Ces opérations sont basées sur l'ordre des éléments et non sur leurs positions dans l'arbre
- Les définitions des fonctions qui suivent sont basées sur l'implantation des arbres sous forme de pointeur

34

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La procédure de l'insertion
 - Si un arbre est vide, alors *Item* est inséré à la racine.
 - Sinon *Item* est inséré soit dans le sous-arbre gauche ou le sous-arbre droit, suivant que *Item* est inférieur ou supérieur à la racine

```
InsertItem(Root, Item)                InsertAtRoot(Root, Item)
{                                     {
    if (!Root) {                     Alouer un nouveau noeud à
        InsertAtRoot(Root, Item);    Root
    } else {                          Root->Item = Item;
        if (Item < Root->Item) {      }
            InsertItem(Root->LChild);
        } else {
            InsertItem(Root->RChild);
        }
    }
}
```

35

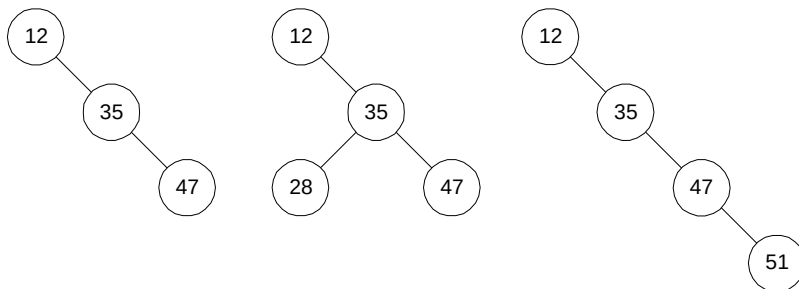
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Supposé l'arbre de recherche binaire de gauche.
- Après l'appel de : InsertItem(Root, 28), on obtient l'arbre du centre.
- Le pire des cas apparaît quand l'arbre est organisé en chaîne, comme l'arbre de droite ici suite à l'insertion de l'élément 51.



36

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La chaîne donne l'ordre croissant des éléments.
- Dans ce cas : $\text{WorstTime} = \text{WorstSpace} = O(N)$
- Dans le cas où l'arbre est parfaitement équilibré ou du moins parfaitement à partir de $h - 1$, on a un nombre d'appels récur­sifs proportionnels à la hauteur de l'arbre, c'est-à-dire $O(\log N)$.

37

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Voici un algorithme itératif pour l'insertion d'un élément. On profite du fait que l'insertion se fait toujours au niveau des feuilles.

```
Insert(Root, Item)
{
    if (!Root) {
        InsertAtRoot(Root, Item);
    } else {
        Child_Root = Root;
        do {
            Parent = Child_Root;
            if (Item < Child_Root->Item) {
                Child_Root = Child_Root->LChild;
            } else {
                Child_Root = Child_Root->RChild;
            }
        } while (Child_Root != NULL);
        if (Item < Parent->Item) {
            InsertItem(Parent->LChild);
        } else {
            InsertItem(Parent->RChild);
        }
    }
}
```

38

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La procédure de suppression
 - Pour supprimer un élément d'un arbre, on doit d'abord trouver sa place dans l'arbre, c'est-à-dire, on doit trouver le sous-arbre dont la racine contient l'élément à supprimer *Item*.

```
DeleteItem(Root, Item)
{
    if (!Root) return; /* Si l'Item à supprimer n'est pas membre de l'arbre */
    if (Item == Root->Item) {
        DeleteRoot(Root);
    } else {
        if (Item < Root->Item) {
            DeleteItem(Root->LChild);
        } else {
            DeleteItem(Root->RChild);
        }
    }
}
```

39

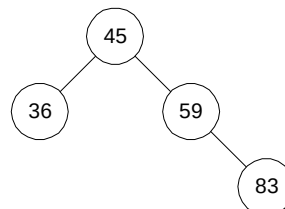
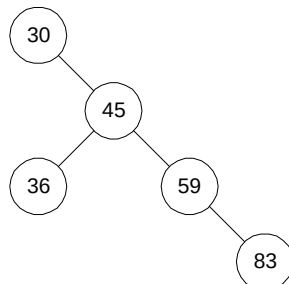
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La procédure de suppression
 - Après suppression de la racine de l'arbre de gauche on obtient l'arbre de droite.



40

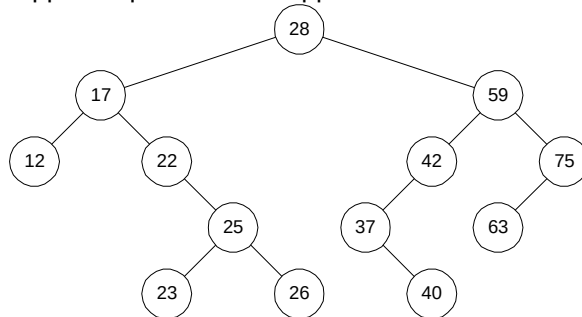
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La procédure de suppression
 - Si les deux sous-arbres sont non vide la procédure est plus complexe. Supposer qu'on désire supprimer la racine de l'arbre suivant :



41

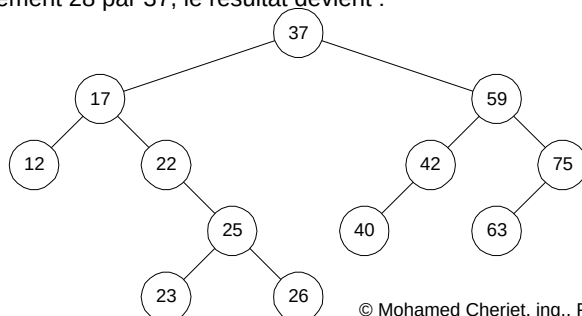
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La procédure de suppression
 - Avant la suppression de l'élément 28, on doit trouver l'élément qui le remplacera. Les seuls éléments adéquats sont le prédécesseur immédiat 26 ou le successeur immédiat 37. Par exemple, si on substitue l'élément 28 par 37, le résultat devient :



42

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Fonctions nécessaires pour supprimer une racine

DeleteRoot(Root)

```
{
    if (!Root->LChild || !Root->RChild) {
        Prune(Root);
    } else {
        DeleteSuccessor(Root->RChild,
                        &Successor);
        Root->Item = Successor;
    }
}
```

Prune(Root)

```
{
    OldRoot = Root;
    if (!Root->RChild) {
        Root = Root->LChild;
    } else {
        Root = Root->RChild;
    }
    free(OldRoot);
}
```

DeleteSuccessor(Root, *Successor)

```
{
    if (!Root->LChild) {
        *Successor = Root->Item;
        Prune(Root);
    } else {
        DeleteSuccessor(Root->LChild,
                        Successor);
    }
}
```

43

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La procédure de recherche d'un élément
 - L'implantation de cet algorithme relève de la définition récursive d'un arbre de recherche binaire.

SearchItem(Root, Item, *Found)

```
{
    if (!Root) {
        *Found = false;
    } else {
        if (Item == Root->Item) {
            *Found = true;
        } else {
            if (Item < Root->Item) {
                SearchItem(Root->LChild);
            } else {
                SearchItem(Root->RChild);
            }
        }
    }
}
```

44

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La procédure de recherche d'un élément
 - Une version itérative peut aussi être implantée.

```
Search(Root, Item, *Found)
{
    *Found = false;
    while (Root && !Found) {
        if (Item == Root->Item) {
            *Found = true;
        } else {
            if (Item < Root->Item) {
                Root = Root->LChild;
            } else {
                Root = Root->RChild;
            }
        }
    }
}
```

45

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Les arbres en épi.
- On commence par une définition récursive de ces derniers :
 - Un arbre en épi T est une arbre binaire complet tel que T est vide ou :
 - Chaque élément du sous-arbre gauche de T est inférieur ou égal à la racine de T
 - Chaque élément du sous-arbre droit de T est inférieur ou égal à la racine de T
 - Les sous-arbres gauche et droit sont des arbres en épi
- L'ordre dans un tel arbre est de haut en bas, mais pas de gauche à droite.
- L'arbre en épi, curieusement structuré, est utile dans plusieurs applications : pour l'implantation des files de priorité et pour l'algorithme de tri HeapSort.

46

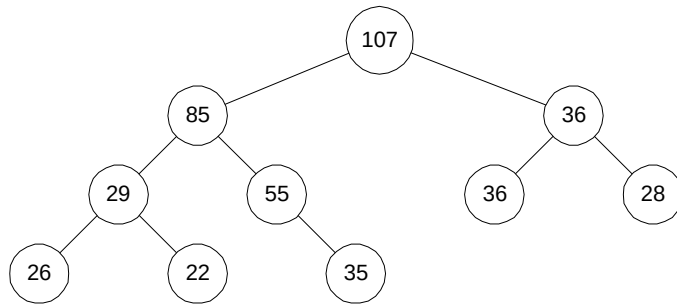
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Exemple d'un arbre en épi



47

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- La majorité des responsabilités globales des TDA arbres binaires sont applicable au TDA arbre en épi.
- Les responsabilités locales sont orientées position, étant donné que la position est une caractéristique significative pour un arbre binaire complet. Afin d'être toujours sûr d'avoir un arbre binaire complet, les insertions et les suppressions sont permises seulement dans la dernière position. C'est-à-dire que chaque élément inséré devient l'élément le plus à droite du plus bas niveau de l'arbre, et que le seul élément qu'on puisse supprimer est l'élément le plus à droite du plus bas niveau de l'arbre.
- N'importe quel élément peut être retrouvé par sa position

48

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Les responsabilités définies :

```
1. Retrieve(Root, *Item, Position)
{
    /* L'élément à la position spécifiée est copié dans
       Item */
}

2. InsertLast(Root, Item)
{
    /* Item est inséré comme une nouvelle feuille, le plus à
       droite possible au plus bas niveau de l'arbre. */
}

3. DeleteLast(Root)
{
    /* L'élément le plus à droite possible au plus bas niveau
       de l'arbre est supprimé */
}
```

49

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- On rajoute deux responsabilités *Swap* qui "*interchange*" les items de *I* et *J* et *ReHeap* qui reconditionne l'arbre de la position *I* à *J*.

```
1. Swap(I, J)
{
    /* Préconditions :
       - J est un descendant de I
       - Le sous arbre dont la racine est en position I
         (incluant la position J) est un arbre en épi, à
         l'exception possible de la position I
       Postconditions :
       - Les éléments des positions I et J sont interchangés; le
         sous-arbre dont la racine est en position J est un
         arbre en épi, à l'exception possible de la position J*/
}

2. ReHeap(I, J)
{
    /* Préconditions: - L'arbre est un arbre binaire complet;
       - L'arbre est en épi à l'exception de I
       Postcondition: Le sous-arbre dont la racine est en
         position I (et dont le dernier élément est
         en position <= J) est un arbre en épi*/
}
```

50

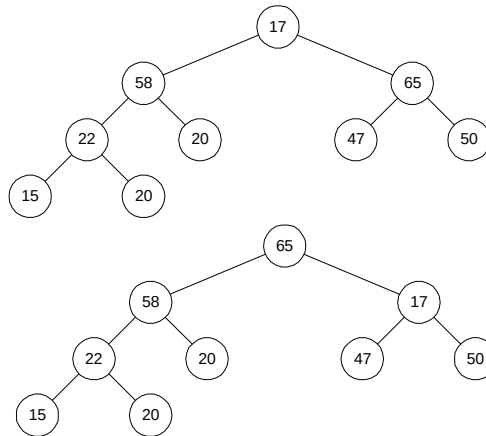
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Si on réalise l'opération $\text{Swap}(1, 3)$ du premier arbre ci-contre, la précondition est satisfaite. On obtient le deuxième arbre qui respecte la postcondition.



51

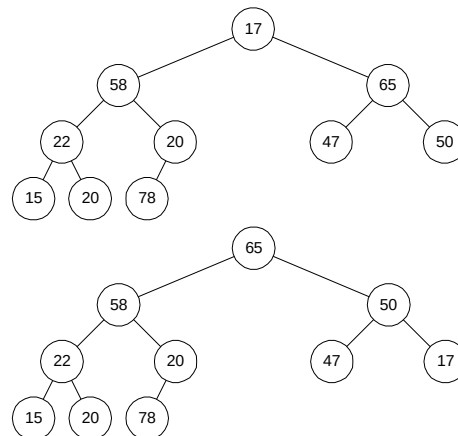
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Si on réalise l'opération $\text{ReHeap}(1, 9)$ du premier arbre ci-contre (qui est en épi à l'exception de la position I et J), la précondition est satisfaite. La postcondition peut être réalisée de plusieurs manières, l'arbre suivant est un arrangement qui est satisfaisant.



52

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Les procédures *Swap* et *ReHeap* sont essentiel dans l'algorithme de tri *HeapSort*.
- Voici un pseudo-code de la fonction *HeapSort* :
for (*i* = last; *i* > 2; *i*--)
 Swap(1, *i*)
 ReHeap(1, *i* - 1)
- À la fin de l'algorithme, le plus grand élément occupe la dernière position et le plus petit élément occupe la première position de l'arbre.

4 Les arbres

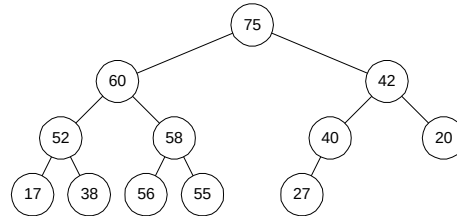
4.6 Les descendants du TDA arbres binaires

- L'organisation en épi d'une TDA file d'attente
- Les files d'attente ou de priorité peuvent-être considérées comme descendantes des arbres binaires en épi.
 - L'ordre est par priorité car l'élément de haute priorité doit être à la racine.
 - Pour les éléments de priorités égales, ils sont ordonnés dans l'ordre chronologique (*first-in, first-out*)
- Pour l'insertion: il est simple d'insérer un item, mais il faut restaurer la priorité d'un arbre en épi.

4 Les arbres

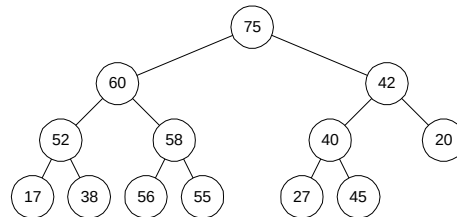
4.6 Les descendants du TDA arbres binaires

- Soit la file d'attente suivante (qui est un arbre en épi)



- On fait appel à *Insert(45)*

Première étape :
on insère 45 en
position 13



55

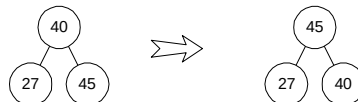
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

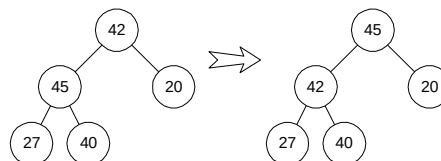
4 Les arbres

4.6 Les descendants du TDA arbres binaires

- On doit maintenant déplacer 45 à sa place parmi ses ancêtres. Pour accomplir ceci, on applique ReHeap à :



- On applique ensuite ReHeap à :



56

GPA665 - Automne 2002

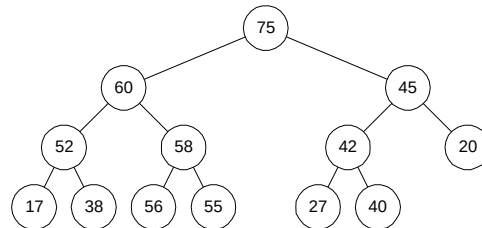
© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Finalement on applique ReHeap à tout l'arbre :

- Ce qui donne comme algorithme :
Insert(Item)
/* N est la position où Item est inséré */
{
 A[N] = Item;
 Root = N/2;
 while (Root >= 1) {
 ReHeap(Root, N)
 Root = Root/2;
 }
}



57

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Pour la suppression: l'élément à supprimer doit être en position 1, alors on l'interchange avec celui de la dernière position, et ensuite on décrémente N. Finalement on restaure la propriété en épi d'un arbre binaire.

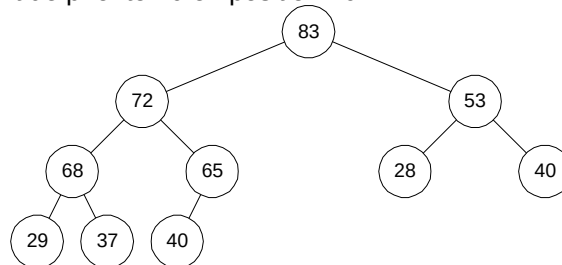
```
DeleteHighest()  
/* N est la position où Item est inséré */  
{  
  Swap(1, N);  
  Decrement(N);  
  ReHeap(1, N);  
}
```

58

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Finalement, on doit considérer le problème de traiter les éléments de même priorité.
- Dans l'exemple suivant, selon l'ordre chronologique, l'élément de priorité 40 en position 7 doit être retrouvé et supprimé avant l'élément de priorité 40 en position 10.



59

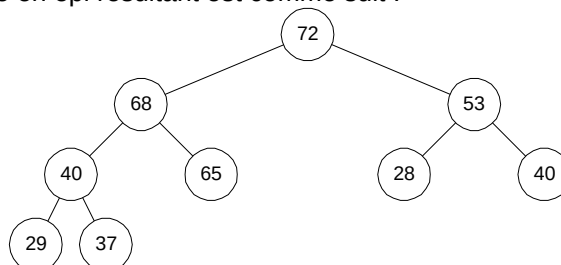
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Pourtant, après l'appel de DeleteHighest, l'élément 83 est supprimé et l'arbre en épi résultant est comme suit :



- Et l'élément de priorité 40 qui était en position 10 est maintenant en position 4. Il devrait alors être retrouvé et supprimé avant l'élément de priorité 40 en position 7.

60

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- On peut éviter ce problème en associant à chaque élément sa priorité et son ordre chronologique. Par exemple, on peut associer une variable *Count* initialisée à zéro et incrémentée à chaque insertion. Au lieu d'insérer simplement un élément, on insère un nœud avec le champ *Item* et le champ *Count*.
- Évidemment, cette variable *Count* doit être considérée seulement pour les cas où il y a deux éléments de même priorité. Ainsi, l'élément de même priorité inséré en premier est supprimé en premier.

61

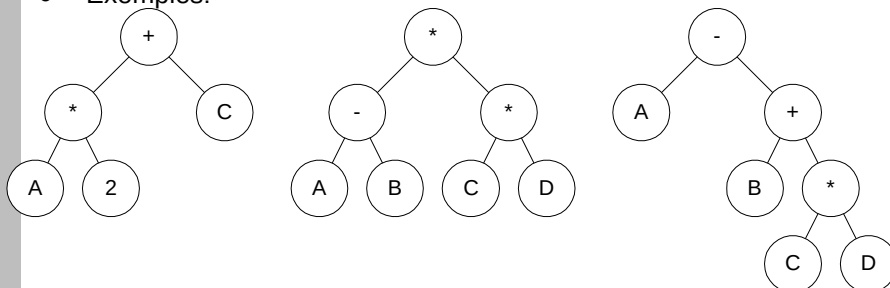
GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Un arbre binaire d'expression est un arbre binaire où chaque nœud non feuille représente un opérateur binaire et chaque feuille est un simple opérande.
- Exemples:



62

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

- En général, le plus loin est un opérateur dans un arbre binaire d'expression, le plus rapidement il est évalué. Deux opérations au même niveau, l'ordre d'évaluation n'est pas important.
- Une des responsabilités d'un arbre binaire d'expression est d'évaluer l'arbre. Ceci induit une responsabilité plus importante: construction d'un arbre binaire d'expression. Comme on l'a déjà mentionné, c'est à partir des expressions écrites en notation préfixée

63

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

Quelques responsabilités du TDA arbre binaire d'expression.

```
Build(RootPrefix)
{
    if (!Prefix.IsEmpty) {
        Prefix.Retrieve(Item, 1);
        Prefix.Delete(1);
        /* Allouer un espace au noeud root, copier Item
           dans champ item de root */

        if (Item.IsAnOperator) {
            Build(Root→Lchild, Prefix);
            Build(Root→Rchild, Prefix);
        }
    }
}
```

64

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

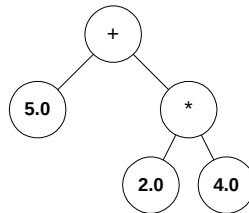
4 Les arbres

4.6 Les descendants du TDA arbres binaires

- Si N référence le nombre d'éléments dans *Prefix*, le temps et l'espace nécessitent $O(N)$.

Exemple: Après l'appel de la procédure *Build()*, le résultat du *Prefix* contenant l'expression **+ 5.0 * 2.0 4.0** est :

:



65

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

4.6 Les descendants du TDA arbres binaires

Evaluate: La valeur réelle de l'évaluation de l'arbre est retournée.

```
EvaluateItem(Root)
{
    if (Root) {
        RetrieveRoot(Item);
        if (ItemIsAnOperator) {
            LeftValue = EvaluateItem(Root→Lchild);
            RightValue = EvaluateItem(Root→Rchild);
            switch(Item.StringValue) {
                case of "+" : return(LeftValue + RightValue);
                case of "-" : return(LeftValue - RightValue);
                case of "*" : return(LeftValue * RightValue);
                case of "/" : return(LeftValue / RightValue);
            }
        } else {
            convert StringValue to RealValue;
            return(RealValue);
        }
    }
}
```

66

GPA665 - Automne 2002

© Mohamed Cheriet, ing., Ph. D.

4 Les arbres

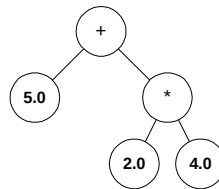
4.6 Les descendants du TDA arbres binaires

Temps est $O(N)$

WorstSpace est $O(N)$

Space est $O(\log N)$ en moyenne.

Exemple: Supposer qu'on ait l'arbre suivant à évaluer:



La valeur retournée est 13,0.